NileGuard

Build Plan — Modules, Milestones and Task Breakdown

Working document derived from `NileGuard_Feature_List.tex` (10 Core + 5 Secondary features, approved)

Team of Four
Faculty of Engineering, Ain Shams University
Computer Engineering and Artificial Intelligence

Cairo, October 2026

Abstract

This document translates NileGuard's approved feature set into four independently buildable modules — Field Node, Gateway & Connectivity, Cloud & Decision Engine, and Application — each with its own sequential task breakdown, its own internal milestones, and an explicit account of which tasks depend on, or must be agreed jointly with, another module. It closes with a quick-reference to every Core and Secondary feature, and with three concrete options for where to draw the line between the GP1 and GP2 deliverables.

Contents

1 Core Features (Quick Reference)

Ten Core features, all of which must ship. Full descriptions, rationale and scope assumptions for each one are in `NileGuard_Feature_List.tex`; this table exists so that everyone reading "C3" or "Core 5" later in this document is looking at the same thing.

Table 1: The ten Core features

#	Name	What it is
C1	Long-range, low-power wireless network	Every field node talks to one central gateway over a long-range, low-power radio link; only the gateway needs an internet connection.
C2	Parametric, agronomically grounded decision engine	Irrigation is decided from a real water-need model — crop type, soil type, growth stage, current conditions — not a fixed schedule or a single hand-set threshold.
C3	Zero-touch node provisioning	A new node powers on, discovers the system, and registers itself with a unique ID and zone — no manual network configuration.
C4	Manual/automatic modes with strict override priority	A person can directly control a valve, and the automatic engine can too; a manual command always wins, so there is exactly one true state per valve.
C5	Fair water allocation under shortage	When the shared pump cannot serve every zone at once, an explicit rule (not first-come-first-served) decides who gets water and for how long.
C6	Local resilience during connectivity loss	If the cloud link drops, the gateway keeps irrigating using a simple local rule until the connection returns.
C7	Swappable battery with power-aware firmware	Field-replaceable batteries, plus firmware that actively manages its own power budget (e.g. sleep cycles).
C8	Web dashboard with stated-reason alerts and recommendations	Live readings, zone/node status, manual control, and every alert or recommendation shown with its plain-language reason.
C9	Basic authenticated access	Login required for the dashboard; a node must authenticate before the gateway/cloud accepts its data or commands.

(continued)

#	Name	What it is
C10	Scalable-by-design architecture	Growing the node count is a configuration change — more rows, more gateways — not a software rewrite.

2 Secondary Features and Where They Belong

Five Secondary features. None of these are a fifth module — each one extends a Core feature above, built inside whichever module already owns that Core feature, and only once that module's Core work is stable. This section is forward-looking: it is what the team needs to think about for GP2, not something being built now.

Table 2: The five Secondary features

#	Name	Home	Extends / description
S1	Disease-risk alerting	Module 3, task CL12; surfaced via Module 4's AP8	Extends **C2**. Flags sustained temperature/humidity conditions known to risk disease (tomato early blight, potato brown rot) using data C2 is already collecting — no new sensor needed.
S2	AI-assisted decision refinement	Module 3, task CL13; surfaced via AP8	Extends **C2**. A non-authoritative suggestion layer on top of the water-balance model — e.g. delaying irrigation when an external forecast says rain is coming.
S3	Advanced security (anomaly detection)	Module 3, task CL14; surfaced via AP8	Extends **C9**. Watches for suspicious traffic patterns — a replayed reading, an implausible timing — beyond the basic login/auth check C9 already provides.
S4	Role-based access	Module 4, task AP7	Extends **C8/C9**. Splits dashboard users into an operator role (monitor + manual control) and an admin role (+ crop/soil config, add hardware). No dedicated Cloud task currently assigned; enforcing it properly will likely need a small addition to CL5's token scopes.
S5	Sensor health and fault detection	Module 3, task CL15; surfaced via AP8	Extends **C2**'s trust in its own input. Flags a sensor as possibly faulty when its readings get stuck or jump by a physically-impossible amount, instead of acting on bad data as if nothing's wrong.

Priority order: S1 → S2 → S3 → S5 ship in that order, each only once Core is verified stable (Module 3's milestone CL-M5, Section **??**). S4 is independent of the other four — it does not block, and is not blocked by, S1/S2/S3/S5 — so it can be scheduled anywhere in GP2 on its own.

3 Cross-Module Contracts

Three things touch more than one module. Agree these in writing before any module starts building — everything else can proceed in parallel once these are fixed.

- **Node ↔ gateway.** Reading packet: `node_id, zone, moisture, temp, humidity, battery_mv, seq`. Command packet: `node_id, valve_state, seq`.
- **Gateway ↔ cloud.** Ingestion payload reuses the reading-packet field set, plus `gateway_id` and `received_at`.
- **Cloud ↔ app.** REST endpoints for readings, zones, nodes, manual overrides, alerts, and auth — shaped once, consumed by the dashboard only.
- **Override rule.** Manual always beats automatic. The same precedence logic is implemented twice on purpose: once in the Cloud module (normal operation) and once in the Gateway module (local fallback) — identical semantics, independent enforcement.

4 Module 1: Field Node — Sensing, Actuation, Power

Owns the node-side half of **C1** (wireless link), all of **C3** (zero-touch provisioning), the node-side half of **C4** (actuation), and all of **C7** (power). Originates the sensor data that C2's decision engine and Secondary features S1 and S5 depend on.

Milestones

ID	Done when	Name	What "done" means
NODE-M1	N1–N2 done	Bench powered up	Every chosen part is wired together and alive — nothing is dead on arrival. No logic yet, just signs of life.
NODE-M2	N3–N5 done	Core behavior proven	A calibrated reading goes out over radio, and a command coming back actually moves the valve and reports its state.
NODE-M3	N6–N9 done	Autonomous on the bench	The full loop runs unattended for 24 hours, survives a battery swap mid-operation, and self-provisions with zero manual config.
NODE-M4	N10–N12 done	Ready at fleet scale	Every physical node is built to spec, battery life is measured in the field (not estimated), and provisioning survives collisions and re-joins.

Task breakdown — build in this order

Task	Builds on	What you'll actually do	Feature
N1	—	Pick and justify every component: a capacitive soil-moisture sensor (doesn't corrode like the resistive kind), a temp/humidity sensor, a low-power MCU, the radio module matching the node↔gateway contract, a relay/MOSFET driver sized for the valve's solenoid current, and a battery + charging circuit. Write down exact part numbers and the reason for each pick — not just a category.	C1, C7
N2	N1	Wire one of every part together on a breadboard and power it on. The only goal is proving nothing is dead on arrival: the MCU boots, each sensor responds to a basic read command, and the relay audibly clicks when driven directly — no custom logic yet.	C7
N3	N2	Record the moisture sensor's raw value in a cup of bone-dry soil and in fully saturated soil to get real wet/dry reference points — don't trust the datasheet's numbers. Write the conversion from raw value to a 0–100% reading, and cross-check temp/humidity against a known-good reference.	C2 input
N4	N3	Write firmware that packages a calibrated reading into the agreed packet format and transmits it over radio on a fixed interval. Confirm with whoever's building the gateway's receiver (G3) that packets arrive intact and the sequence number increments correctly.	C1
N5	N2	Write firmware that listens for an incoming command packet, drives the relay to open or close the valve, and sends back a confirmation packet with the state it actually reached — not just the state it was told to reach, so a stuck valve can be detected later.	C4
N6	N4, N5	Implement the sleep/wake cycle between readings, and measure the actual current draw in each state with a multimeter or power profiler — not a datasheet estimate. This number is what N11's field test later validates or corrects.	C7
N7	N6	Make battery removal/replacement safe: the node must not corrupt its saved ID, zone, or sequence number if power is cut mid-operation, and must resume cleanly on next boot. Test by physically pulling the battery mid-transmission, repeatedly.	C7
N8	N7	On first boot with no saved ID, broadcast a provisioning request, wait for an assigned ID/zone back, and persist it to flash so it survives a reboot or battery swap. Agree the exact handshake message with whoever is building G7 and CL6 (joint task) before writing this.	C3

(continued)

Task	Builds on	What you'll actually do	Feature
N9	N4–N8	Leave one fully-assembled node running its complete loop — sense, send, wait for a command, actuate, sleep — unattended for a full 24 hours. First proof everything built so far works together continuously, not just in isolated tests. Log anything that stalls, resets, or drifts.	C1, C4, C7
N10	N9	Build, wire, flash, and calibrate every remaining physical node to the spec proven in N9. Expect per-unit variance (slightly different wet/dry readings) — re-run N3's calibration per unit rather than assuming they're identical.	C1, C7
N11	N10	Deploy a node in real field conditions, not on the bench, and let it run for several days on one charge, logging actual runtime. Use the result to correct N6's sleep-interval numbers — temperature and real radio range change power draw.	C7
N12	N10	Deliberately break N8's happy path: force two nodes to request an ID at the same moment, reset a node and have it re-join, swap a node's hardware and confirm it keeps or correctly re-requests its identity — all without an operator manually fixing state.	C3

5 Module 2: Gateway and Connectivity — the Single Aggregation Point

Owns the gateway-side half of **C1**, the provisioning-relay half of **C3**, all of **C6** (local resilience), and the edge-enforcement half of **C9** (authenticated access).

Milestones

ID	Done when	Name	What "done" means
GW-M1	G1–G2 done	Testable without real hardware	The gateway boots and can be driven entirely by synthetic traffic, so this module never has to wait on Module 1.
GW-M2	G3–G6 done	Real traffic path proven	A real node's packet is received, parsed, forwarded to the cloud, and a command makes it back down to the right node.
GW-M3	G7–G8 done	Thinks for itself	A new node can provision itself through the gateway, and the gateway keeps irrigating alone with no cloud link.
GW-M4	G9–G12 done	Resilient and secure at scale	Fallback engage/recover is proven under a real pulled link, unauthenticated traffic is rejected, and the registry holds at full node count.

Task breakdown — build in this order

Task	Builds on	What you'll actually do	Feature
G1	—	Set up the physical gateway: an SBC (a Pi Zero 2 W or a repurposed laptop does this job fine — a full Pi 4 is overkill) plus the radio receiver matched to the node radio, with a base OS image that can run your code on boot.	C1
G2	G1	Build a small script that generates fake reading packets in the exact node↔gateway format and feeds them in as if received over real radio. This exists so this module never has to wait on Module 1's physical hardware — prove every later task against mock traffic first.	C1
G3	G1, G2	Write the firmware/driver that actually listens on the radio, frames incoming bytes into packets, and parses them into the agreed fields. Validate against the mock harness (G2) first, then against a real transmitting node (N4) once it exists.	C1
G4	G3	Forward a parsed reading to the Cloud module's ingestion endpoint (CL1), adding the `gateway_id` and a `received_at` timestamp the node itself doesn't know. Queue and retry if the cloud is briefly unreachable — don't silently drop readings.	C1
G5	G4	Receive a valve command from the cloud and forward it down to the correct node over radio, using the node registry (G6) to know which physical node to address.	C4 support

(continued)

Task	Builds on	What you'll actually do	Feature
G6	G3	Keep a local table of every node this gateway has heard from: node ID, zone, last-seen time. This is what G5 looks up to route a command, and what G9 later checks to see whether a node has gone silent.	C3
G7	G6	Listen for a node's first-boot broadcast, forward the registration request to the cloud, get back an assigned ID/zone, and relay that answer to the node. Agree the message shape with whoever's building N8 and CL6 (joint task) first — it's a three-way contract.	C3
G8	G6	Implement a simple moisture-threshold decision (below X% $\rightarrow$ open valve) that runs entirely on the gateway's own hardware, with no dependency on the cloud being reachable. Deliberately dumber than the cloud's engine — its only job is keeping irrigation alive during an outage.	C6
G9	G4, G8	Continuously check whether the cloud uplink is actually working (not just "no error thrown"), automatically switch into G8's fallback mode when it isn't, then switch back and resync cleanly once the link returns.	C6
G10	G6	Reject any packet claiming to be from a node ID the gateway hasn't been told to trust, checked against the auth token CL5 issues. Closes the gap where anyone with matching radio hardware could inject fake readings or commands.	C9
G11	G10, N10	Confirm the registry and forwarding path hold up once every physical node (not 1–2 test units) is transmitting: check for lookup slowdowns, dropped packets under concurrent traffic, and registry entries that never expire for nodes gone for good.	C1, C10
G12	G9, G11	Physically pull the cloud link while the gateway is running and confirm G9's fallback engages within a reasonable time, irrigation continues under G8's rule, and the gateway resyncs its queued data cleanly once the link returns.	C6

6 Module 3: Cloud and Decision Engine — the System's Reasoning

Owns **C2** (decision engine), **C5** (fair allocation), the backend half of **C9** (auth), all of **C10** (scalable architecture), and every backend half of the Secondary features (S1, S2, S3, S5).

Milestones

ID	Done when	Name	What "done" means
CL-M1	CL1–CL2 done	Data has somewhere to go	Ingestion accepts a real reading and the CRUD API for devices/zones/users works against a real database.
CL-M2	CL3–CL5 done	First real recommendation	The decision engine returns an actual recommendation from seeded crop data, and a user can log in and get a valid session.
CL-M3	CL6–CL8 done	Vertical slice proven	A node provisions itself through the cloud, manual override always wins, and the four-module smoke test passes.
CL-M4	CL9–CL11 done	Reasons properly, holds at scale	The real water-balance model is live, fair allocation is tested under shortage, the architecture is load-tested and documented.
CL-M5	CL12–CL15 done	Secondary intelligence shipped	Disease-risk, AI refinement, anomaly detection and sensor-health are all live, added one at a time in that order.

Task breakdown — build in this order

Task	Builds on	What you'll actually do	Feature
CL1	—	Build the endpoint the gateway posts readings to, and the time-series table it writes into: `node_id, timestamp, moisture, temp, humidity, gateway_id, received_at`. The single entry point all historical data flows through — get this schema right before anything else is built on it.	C1 consumer
CL2	CL1	Model devices (nodes), zones, and users as proper database tables with relationships — a node belongs to a zone, a user can see some zones — and expose CRUD endpoints for each. The backbone Module 4 talks to for everything that isn't a live sensor reading.	C10
CL3	CL2	Seed the database with real moisture/temp/humidity ranges and growth-stage curves, starting with tomato and potato. Keep the schema general enough to add more crops later, even though only tomato gets full calibration and evaluation this cycle.	C2
CL4	CL3	Write the first, deliberately simple recommendation logic: compare a zone's current moisture against its crop's threshold and recommend irrigate/don't-irrigate. This is what gets demoed first — CL9's water-balance model replaces it later, it doesn't need to be clever yet.	C2
CL5	CL2	Implement hashed-password user login that issues a session/token, plus a separate token type the gateway uses to prove a node's identity — the one G10 checks against.	C9
CL6	CL2, CL5	When a new node's registration request arrives via the gateway, assign it a unique ID/zone and record it in the device table. Agree the message format with whoever's building N8 and G7 (joint task) — same three-way contract as G7.	C3
CL7	CL4, CL5	Enforce exactly one source of truth per valve, and that a manual command always wins over an automatic one — if a farmer just turned a valve off, the decision engine doesn't get to turn it back on a minute later because moisture dropped.	C4
CL8	CL1–CL7	Run one real reading all the way from a physical node through the gateway into this pipeline, get a recommendation out, and confirm a manual override sent from here reaches the node and wins. Done together with Modules 1, 2 and 4 — the first time all four are proven against each other for real.	All modules
CL9	CL8	Replace CL4's simple threshold with the full FAO-56 ET_0/ET_c water-balance model: compute each zone's actual water need from its crop's growth stage (derived from planting date) and recent weather, not just a fixed moisture cutoff.	C2
CL10	CL9	Handle multiple zones wanting water at once when the shared pump/supply can't serve all of them: detect the conflict and apply a rule (e.g. driest-zone-first, or proportional sharing) instead of first-come-first-served. Test against synthetic concurrent-demand scenarios.	C5
CL11	CL9, CL10	Review the architecture and data model and load-test at rising simulated node counts, then document that growing the deployment is a config change — more rows, more gateways — not a rewrite. This is the evidence behind the "scalable, plug-and-play" claim.	C10
CL12	CL11	Add a rule flagging disease risk from sustained humidity/temperature thresholds already being collected — e.g. tomato early blight or potato brown rot conditions sourced from the team's literature. Free in the sense that no new sensor is needed, only new logic on existing data.	S1
CL13	CL12	Pull in an external weather forecast and use it to adjust CL9's recommendation ahead of time (e.g. skip watering if rain is coming), framed explicitly as a suggestion layer on top of the water-balance model, not a replacement for it.	S2
CL14	CL13	Look for suspicious patterns in node traffic — the same reading replayed, or values changing faster than physically possible — and flag them as potential security events on the backend side. Module 4's AP8 surfaces these in the UI.	S3
CL15	CL14	Separately from security, flag a sensor as possibly faulty when its own readings get stuck on one value or jump by a physically-impossible amount between two consecutive readings — a hardware-health signal, not a security one.	S5

7 Module 4: Application — the Farmer-Facing Dashboard

Owns **C8** (dashboard), the login-UI half of **C9**, and **S4** (role-based access). Surfaces S1, S2, S3 and S5 as each ships from Module 3.

Milestones

ID	Done when	Name	What "done" means
AP-M1	AP1 done	Logged in, empty shell	A real user can log in against the real backend and land on a working, empty dashboard.
AP-M2	AP2–AP4 done	Monitor and control loop works	Live readings, manual override, and recommendations-with-reasons are all visible and working.
AP-M3	AP5 done	Proven against the real stack	Verified end-to-end with no mocks anywhere — real backend, real gateway, real node.
AP-M4	AP6–AP8 done	Full farmer experience	History, roles, and every secondary alert are visible as each one ships from Module 3.

Task breakdown — build in this order

Task	Builds on	What you'll actually do	Feature
AP1	CL5	Stand up the dashboard project and build the login screen wired to the real auth API, not a mock — the first proof that the frontend and backend contracts actually match.	C9
AP2	AP1	Show per-zone moisture/temperature/humidity and whether each node is online or offline (derived from last-seen time), pulling from the readings API.	C8
AP3	AP2	Add a per-zone valve on/off control that calls CL7's override-arbitration endpoint, and show the farmer the valve's actual confirmed state from N5 — not just the button they clicked.	C4, C8
AP4	AP3	Surface the decision engine's output with its plain-language reason — e.g. "Zone 2: irrigate, moisture at 18%, below tomato's 25% threshold" — so a non-technical farmer understands why the system is suggesting what it's suggesting.	C8
AP5	AP4	Run the dashboard against the real backend, a real gateway, and a real node — not mocked data anywhere — covering login, live readings, manual override, and the alerts panel in one pass.	All modules
AP6	AP5	Add time-series charts per zone so a farmer can see moisture/temperature trends over days or weeks, not just the current instant.	C8
AP7	AP6	Split the UI into an operator role (monitor + manual control only) and an admin role (adds crop/soil configuration and registering new hardware), enforced by what the auth token actually permits — not just hidden buttons.	S4
AP8	AP7	As each Cloud secondary feature ships (CL12–CL15), add its alert type to the AP4 panel — disease-risk, AI suggestion, security flag, sensor-fault flag — one at a time, in the same order Module 3 ships them.	S1, S2, S3, S5

8 How the Modules Connect

The contracts in Section 3 fix the wire formats. This is the other half: which specific tasks, across which modules, have to be built or agreed together — so nobody finds out at integration time that their part doesn't match a teammate's. Ordered by increasing task number, module by module (Field Node first, then Gateway, then Cloud).

1. **Node → gateway radio link** (`N4` ↔ `G3`, via `G2`). N4 isn't "done" until a real packet it sends is actually received and parsed by G3 — that's one joint checkpoint, not two separate unit tests. Until N4 exists, G3 is validated against G2's mock traffic instead, so Module 2 is never blocked waiting on Module 1's hardware.

2. **Node → gateway command delivery** (`N5` ↔ `G5`, `G6`). G5 forwards a command to a specific node using G6's registry to find it; N5 is what actually receives that command, drives the relay, and reports back. Confirm the full round trip together, not each side in isolation.

3. **Provisioning handshake** (`N8`, `G7`, `CL6`). A three-way contract, not three separate tasks. A new node's first-boot broadcast only works if the node's broadcast format (N8), the gateway's relay format (G7), and the cloud's registration response (CL6) all agree on the same message shape. Settle this one in a joint conversation before anyone writes code against it.

4. **Full fleet → gateway scale test** (`N10 → G11`). G11's concurrency/scale testing is meaningless without Module 1 having actually built out the full physical node count first. Sequence this across modules, not just within each one.

5. **Gateway → cloud ingestion** (`G4 → CL1`). Two ends of one pipe. CL1's schema has to exist before G4 has anywhere real to send readings to — agree the field list once, per the gateway↔cloud contract above, and don't let either side drift from it independently.

6. **Cloud auth → gateway enforcement** (`CL5 → G10`). G10 can't reject unauthenticated node packets until CL5 is actually issuing the node-auth tokens it checks against. Build CL5 first, or stub a fake token for G10 to test against in the meantime.

7. **Cloud recommendation → app display** (`CL4/CL9 → AP4`). AP4 just displays whatever the decision engine computes — first CL4's threshold, later CL9's water-balance output. AP4 does not have logic of its own; if Cloud changes the response shape, AP4 needs updating in lockstep.

8. **Cloud auth → app login** (`CL5 → AP1`). AP1's login screen is only a real test once it's wired to CL5's actual auth API, not a placeholder form. If CL5 slips, AP1 should still build against a short-lived mock — just don't call AP1 "done" until the swap to the real API happens.

9. **Cloud override arbitration → app control** (`CL7 → AP3`). AP3's override button is a thin UI layer over CL7's arbitration logic — same lockstep-update caveat as the recommendation display above.

10. **The vertical-slice smoke test is one event, not two** (`CL8 ↔ AP5`). CL8 and AP5 are the same end-to-end run, watched from the backend side and the dashboard side respectively — and both implicitly require Module 1 and 2 to already be at their own "autonomous" / "thinks for itself" milestones. Schedule it as one team session with all four people present, not two separate check-offs on different days.

11. **Secondary features → their alerts** (`CL12–CL15 → AP8`). Each Cloud secondary only becomes visible to a farmer once AP8 adds its matching alert type. Ship and surface them in the same order Module 3 ships them — disease-risk, then AI refinement, then anomaly detection, then sensor-health — don't get ahead of whichever hasn't landed yet.

9 Team Allocation: Two Staffing Scenarios

The plan above is written per module, independent of who builds it. Here are two concrete ways the team of four can staff it, with a recommendation for coordinating either way.

Scenario 1 — One Owner per Module

Each of the four team members owns one module start to finish. This is the simplest arrangement, and matches the default assumption used throughout this document. For each module, here is what that person will have genuinely learned by the end of it, and the specific stack they need to pick up to get there.

Table 11: Scenario 1: per-module learning outcomes and required stack

Module	What you'll learn	Stack to acquire
Field Node	Embedded firmware development, sensor interfacing and calibration, low-power design (sleep/wake budgeting), radio communication at the link layer, and basic electronics and battery management.	A systems-level embedded language (typically C/C++ on whatever microcontroller toolchain the chosen board uses), the radio library that ships for the chosen radio module, a multimeter for real power measurement, basic breadboarding/soldering, and firmware version control.
Gateway & Connectivity	Linux system administration on an SBC, radio receiver/driver integration, building a small always-on service, local decision-rule logic, and resilience/failover engineering.	A Linux-based OS on the gateway hardware, a general-purpose backend language the team is already comfortable with (any one works — Python, JavaScript, or similar), a lightweight local datastore for the node registry, and that language's HTTP client for talking to the cloud.

(continued)

Module	What you'll learn	Stack to acquire
Cloud & Decision Engine	Backend API design, relational data modeling, authentication fundamentals, time-series data handling, an agronomic water-balance model (FAO-56), and fairness/scheduling logic.	A backend web framework in whichever language the team knows best, basic relational-database and schema/ORM skills, a token- or session-based auth library, basic numerical methods for the FAO-56 model, and a load-testing tool for CL11 (any mainstream one works).
Application	Frontend web development, consuming REST APIs, live/polling state management, role-based UI design, data visualization, and writing UX copy for explainable alerts.	A frontend framework or library of the team's choice, any common charting library for AP6, the ability to call a REST API from the client, and basic token-based auth handling on the frontend side.

Scenario 2 — Two Pairs, Two Modules Each (Parallel)

Instead of one owner per module, split the team into two pairs of two and put both people in a pair on the same module at the same time. This trades one slower, more thoroughly-understood build per module for faster progress and fewer solo dead-ends — someone to unblock you is always sitting next to you.

Recommended pairing: Pair 1 takes **Field Node + Gateway**; Pair 2 takes **Cloud + Application**. Both pairs work their two modules in parallel from day one — this is not "Pair 1 finishes, then Pair 2 starts"; all four modules move at once, exactly as the base plan intends.

This specific pairing is not arbitrary. Of the eleven cross-module connections in Section **??**, seven stay entirely inside one pair's two modules — every node↔gateway task, and every cloud↔app task. Only four touch the boundary between the pairs: the three-way provisioning handshake (`N8`, `G7`, `CL6`), gateway→cloud ingestion (`G4→CL1`), cloud auth→gateway enforcement (`CL5→G10`), and the whole-team vertical-slice smoke test (`CL8↔AP5`). Pairing this way turns most integration work into a conversation between two people sitting together, and leaves only four deliberate sync points between the pairs instead of constant cross-team coordination.

Assigning people to pairs: put whoever already leans embedded/hardware on Pair 1 and whoever leans web/backend development on Pair 2 — this split exists to match the pairing to the team's real skill split, not to divide the team randomly.

How the two pairs work together:

- **Within a pair:** sit together (or screen-share continuously) for the harder, newer parts of each task; split off to work solo on well-understood, independent sub-steps (e.g. one person wires a sensor while the other writes the calibration script) and recombine for the next joint checkpoint.
- **Between pairs, routine:** a short (10–15 minute) sync twice a week is enough — there is no constant dependency, so a standing meeting longer or more frequent than that is wasted time.
- **Between pairs, on the four boundary items above:** schedule each one as an explicit joint session with the specific people on both sides of that one connection, not the whole pair-to-pair sync — e.g. the provisioning handshake only needs whoever is writing N8/G7 from Pair 1 and CL6 from Pair 2 in the same room, not all four people.
- **The vertical-slice smoke test (CL8/AP5)** is the one point where both pairs' work has to be in the same room at the same time — schedule it as a full-team session once Pair 1 has reached NODE-M3/GW-M3 and Pair 2 has reached CL-M2, not before.

10 Where to Draw the GP1/GP2 Line

Each module now has its own build order and its own milestones, independent of any GP1/GP2 split. Below are three defensible ways to decide, per module, which task to stop at for the GP1 deadline — pick one, don't average them.

Option	Field Node	Gateway	Cloud	Application
A — Minimum defensible slice	through N9	through G8	through CL8	through AP5
B — Balanced (recommended)	through N10	through G9	through CL8	through AP6
C — Front-loaded	through N12	through G11	through CL11	through AP7

Option A — Minimum defensible slice
Stops exactly at each module's "proven once, end-to-end" milestone. Lowest hardware risk — doesn't require the full physical fleet to exist yet. Best if procurement or hardware assembly is running behind.

Option B — Balanced (recommended)
Pulls forward the parts of GP2 that are low-risk and high demo value — the full physical fleet, automatic fallback switching, trend charts — while leaving the genuinely hard work (the FAO-56 rewrite, fair allocation, security hardening) for GP2, where it belongs.

Option C — Front-loaded
Effectively finishes all ten Core features in GP1, leaving GP2 purely for the four ordered Secondary features and final evaluation/defense. High payoff if the team is ahead of schedule, but front-loads real risk: the water-balance model and fair-allocation algorithm are two of the hardest, least-proven pieces of work, now due at the GP1 deadline instead of GP2's.

Closing Notes

- **On pairing.** Each module is sized for one person as a baseline, but any module can take two — pair up on Modules 1 and 2 around each module's "real traffic / vertical slice" milestone if hardware debugging is the bottleneck, then split back once it's proven.
- Secondary features are never a fifth module — each one extends the Core feature it's built on top of, inside the module that already owns that feature, and only after that module's Core work is stable.